

PERPScope · MARKET ANALYTICS

ETL Pipeline

Build-It-Yourself Manual

A complete, phase-by-phase manual for the automated ETL pipeline powering PerpScope's **Market Analytics** feature (React app) and a **Power BI** dashboard. Every phase pairs the **why**, the concrete **how**, and a **Definition of Done**.

STACK

Python extraction · **BigQuery** warehouse · **dbt** transformations · **GitHub Actions** orchestration (CI/CD) · dimensional modeling · **Power BI**

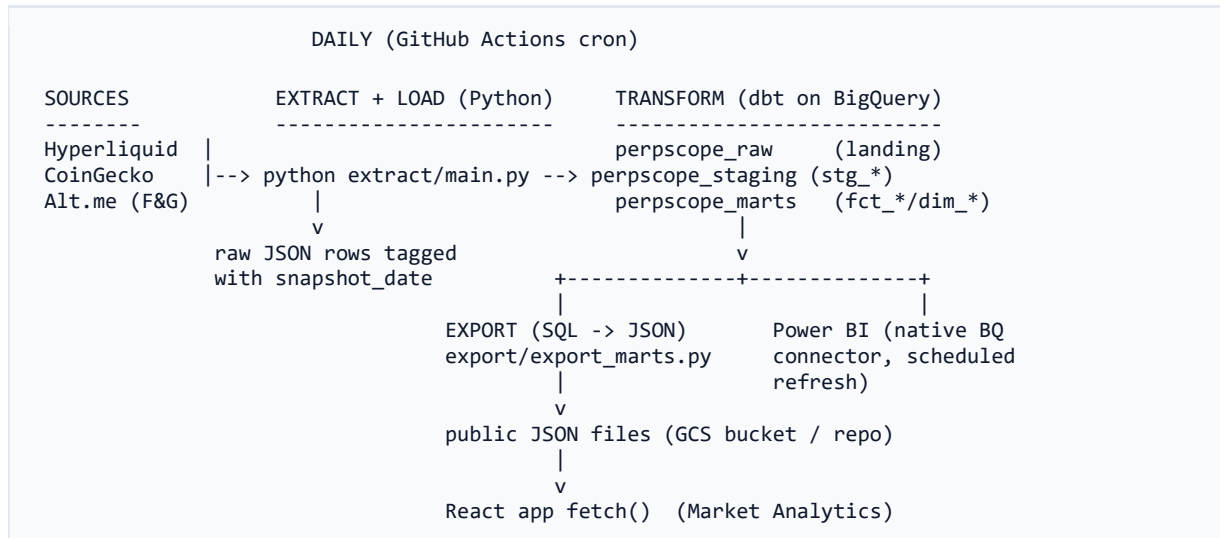
Audience: a data-analytics-engineer résumé piece.

Contents

Contents	2
1. The 10,000-ft Picture	4
Why this shape (ELT, not ETL)	4
2. Tech Stack & Why Each Piece	5
3. The Three Dashboards We Are Building Toward	6
Dashboard 1 — Market Overview	6
Dashboard 2 — Capital Rotation (Open Interest)	6
Dashboard 3 — Smart Money Analytics (top 100 traders, 30d)	6
4. Repository / Folder Layout	7
5. Data Architecture: raw → staging → marts	8
5.1 perpscope_raw — the landing zone	8
5.2 perpscope_staging — clean, typed, renamed (dbt views)	8
5.3 perpscope_marts — the analytics layer (dbt tables)	8
6. Phase-by-Phase Build Roadmap	9
Phase 1 — Google Cloud & BigQuery Setup	10
Phase 2 — Extract (Python)	11
2.1 The sources & endpoints	11
2.2 Extractor contract (every extractor follows this)	11
Phase 3 — Load into BigQuery (raw)	13
3.1 Idempotency strategy: WRITE_TRUNCATE per partition	13
3.2 The generic loader	13
Phase 4 — Transform with dbt (staging → marts)	14
4.1 Project config	14
4.2 Declare sources (with freshness checks)	14
4.3 Staging example (dedupe to latest load per day, cast, rename)	15
4.4 The spine: int_asset_daily	15
4.5 Facts — clean, reusable, all-assets/all-days (no top-N)	15
4.6 Tests	16
Data Dictionary — What the Fact Tables Look Like	18
dim_asset — the asset dimension	18
fct_asset_metrics_daily — the wide per-asset fact	19
fct_fear_greed — the two thin Dashboard-1 facts	21
fct_asset_positioning_daily — smart-money positioning	22
fct_trader_daily — the two trader-level facts	23
Chart → table map	24
Phase 5 — Orchestrate Daily with GitHub Actions	25

Phase 6 — Serve to the React App (static JSON export)	27
Phase 7 — Power BI Dashboard	28
8. Data Quality, Testing & Monitoring	29
9. Cost & Free-Tier Limits	29
10. Appendix: the Metric / Data Dictionary	29
How to use this manual from here	29

1. The 10,000-ft Picture



One sentence: every day a scheduled job pulls fresh market data, drops it **raw** into BigQuery, dbt cleans and reshapes it into **fact/dimension tables**, and the finished marts feed two consumers — a **Power BI** dashboard (live SQL connection) and the **React app** (via a nightly JSON export).

Why this shape (ELT, not ETL)

We **Extract, Load** raw first, then **Transform** **inside the warehouse** with SQL/dbt. This is the modern pattern (ELT) and it's what BigQuery + dbt are built for. We still call the folder **ETL** because that's the umbrella term you used, but architecturally it's ELT.

Worth a sentence in your résumé: *"Built an ELT pipeline (Python → BigQuery → dbt)."*

2. Tech Stack & Why Each Piece

Layer	Tool	Why (and the résumé line it earns you)
Extraction	Python 3.11 + requests	Universal for API pulls. Simple, testable.
Data warehouse	Google BigQuery (sandbox/free)	Serverless, 10 GB storage + 1 TB query/month free — our data is tiny. Columnar, SQL-native.
Load	google-cloud-bigquery client	Loads newline-delimited JSON straight into partitioned raw tables.
Transform	dbt Core (dbt-bigquery)	The analytics-engineering tool: layered SQL models, tests, docs, lineage graph. The strongest résumé signal.
Orchestration	GitHub Actions (cron)	Free, YAML, lives in the repo. Demonstrates CI/CD + scheduling.
Serving → app	Static JSON export to public GCS bucket	Keyless & free; matches how PerpScope already fetches data client-side.
Serving → BI	Power BI Desktop/Service	Native BigQuery connector, scheduled refresh.
Version control / secrets	Git + GitHub Secrets	Service-account key stored as an encrypted secret, never in code.

A note on "free": BigQuery **sandbox** (no billing account attached) is genuinely free but tables get a 60-day expiration and you can't use streaming inserts. We avoid streaming (we use batch load jobs) and set explicit partition expirations, so sandbox is fine for a résumé project. If you attach a billing account you stay inside the free tier anyway at this data volume.

3. The Three Dashboards We Are Building Toward

Every mart table exists to feed one specific chart. Keep this mapping in mind — it's the "north star" for the whole warehouse. Charts don't get their own tables: each is a **sort/filter of a clean fact**. Six facts + **dim_asset** feed all ten charts.

Dashboard 1 — Market Overview

Chart	Fact table	Client does
BTC market regime (bull/bear)	fct_btc_regime	latest row (HMM label + confidence)
Fear / Greed index + history	fct_fear_greed	latest + time series
Hottest 5 coins today	fct_asset_metrics_daily	sort vol_change_24h_pct desc, take 5
Strongest 5 coins (30d)	fct_asset_metrics_daily	sort rs_30d desc, take 5

Dashboard 2 — Capital Rotation (Open Interest)

Chart	Fact table	Client does
Treemap: OI dominance	fct_asset_metrics_daily	oi_dominance (area) + oi_change_24h_pct (color)
Top 5 OI increases (7d)	fct_asset_metrics_daily	sort oi_change_7d_usd desc, take 5
Bar: highest OI/mcap ratio	fct_asset_metrics_daily	sort oi_mcap_ratio desc, take 5

Dashboard 3 — Smart Money Analytics (top 100 traders, 30d)

Chart	Fact table	Client does
Top 100 trader stats + positions	fct_trader_daily (+ fct_trader_positions)	filter in_cohort, sort pnl_30d_usd desc
Net positioning by asset (stacked bar)	fct_asset_positioning_daily	sort pct_long desc, take 10
Daily inflow/outflow (top 10 by mcap)	fct_asset_positioning_daily	filter market_cap_rank ≤ 10, show inflow_usd

4. Repository / Folder Layout

Everything ETL lives in **PerpScope/ETL/**, **except** the GitHub Actions workflow, which *must* live at the repo root under **.github/workflows/** (GitHub only reads workflows there).

```

PerpScope/
|-- .github/
|   |-- workflows/
|       |-- market_analytics_etl.yml    # Phase 5 - daily cron
|-- ETL/
|   |-- README.md                      # <- this manual
|   |-- docs/
|       |-- DATA_DICTIONARY.md        # every metric's exact definition
|   |-- extract/                       # Phase 2 & 3 - Python E+L
|       |-- extractors/
|           |-- hyperliquid.py          # OI/vol/funding, candles, leaderboard
|           |-- coingecko.py            # market caps, top-by-mcap universe
|           |-- feargreed.py            # Alternative.me F&G index
|           |-- load_bigquery.py        # generic raw-JSON -> BigQuery loader
|           |-- main.py                 # runs all extractors, then loads
|           |-- config.py               # dataset names, asset universe
|           |-- requirements.txt
|   |-- dbt/                           # Phase 4 - transformations
|       |-- dbt_project.yml
|       |-- profiles.yml
|       |-- packages.yml               # dbt_utils
|       |-- models/
|           |-- staging/                # stg_* (1:1 with raw, views)
|           |-- intermediate/           # int_asset_daily, int_trader_positions
|           |-- marts/                 # dim_asset + 6 fct_* tables
|   |-- export/                        # Phase 6 - marts -> static JSON
|       |-- export_marts.py
|-- src/features/market/               # (React side, built later)
|-- data/                             # OR fetched from public GCS bucket

```

5. Data Architecture: raw → staging → marts

This is the heart of "analytics engineering." Three BigQuery **datasets**, three purposes.

5.1 perpscope_raw — the landing zone

- **Purpose:** land API responses **exactly as received**, with minimal parsing. Never transformed. This is your audit trail and lets you re-run transformations without re-hitting APIs.
- **Bookkeeping columns on every raw table:**
 - `snapshot_date` (DATE) — the logical day of the pull; **the partition key**.
 - `_loaded_at` (TIMESTAMP) — when the row was loaded.
 - `payload` (JSON/STRING) — the raw object, plus fields we flatten for convenience.
- **Partitioning:** partition by `snapshot_date` so every query and dbt incremental model scans only the days it needs.
- **Tables:** `raw_hl_asset_ctxs`, `raw_hl_candles`, `raw_hl_leaderboard`, `raw_hl_positions`, `raw_cg_markets`, `raw_feargreed`.

5.2 perpscope_staging — clean, typed, renamed (dbt views)

- **Purpose:** one staging model per raw table. **1:1** with the source — no joins, no business logic. Cast types, rename to snake_case, deduplicate to the latest load per day, parse the JSON payload into real columns.
- Materialized as **views** (cheap, always fresh).
- Naming: `stg_<source>_<entity>`.

5.3 perpscope_marts — the analytics layer (dbt tables)

- **Dimensions** (`dim_asset`) — descriptive attributes (symbol, name, is_stablecoin...).
- **Facts** (`fct_*`) — clean, insightful, **reusable** tables at all-assets/all-days grain.
- Materialized as **tables** (fast for BI/JSON export). Small enough to fully rebuild daily; heavy ones can switch to **incremental** later.
- **Intermediate `int\_` models** sit between staging and facts to hold reusable building blocks — most importantly `int_asset_daily`, the **one-row-per-asset-per-day** spine.

Facts store the full population; the client sorts and slices. A fact holds **every** asset at its grain plus every column a component might sort or filter on — **no `rank`, no `LIMIT`, no top-N in the warehouse**. React and Power BI apply **ORDER BY** / **LIMIT** themselves. Only **definitional** filters (e.g. the top-100-trader cohort, via an `in_cohort` flag) live in the warehouse. That's why 10 charts need just **6 facts**.

The dependency flow (dbt builds this lineage graph for you)

```
raw --> stg_* (views) --> int_asset_daily -----+
                                     int_trader_positions +--> fct_* (tables) --> JSON / Power BI
                                     dim_asset -----+
```


6. Phase-by-Phase Build Roadmap

Build in this order. **Each phase is independently testable** — you'll prompt me to do them one at a time, and we'll confirm the Definition of Done before moving on.

Phase	What you get	Deps	Prompt me with
1. Cloud setup	GCP project, BigQuery datasets, service account key	—	"Do Phase 1"
2. Extract	Python extractors that print clean records from every API	1	"Build the extractors"
3. Load	Raw tables populated in perpscope_raw	1, 2	"Build the BigQuery loader"
4. Transform	dbt project: staging → int → 6 reusable facts	3	"Build the dbt models"
5. Orchestrate	GitHub Actions runs the whole thing daily	2–4	"Wire up GitHub Actions"
6. Serve → React	Nightly JSON export the app can fetch	4, 5	"Build the JSON export"
7. Power BI	Dashboard connected to BigQuery marts	4	"Help me build the Power BI dashboard"

We do NOT build the React UI in this manual — that's a separate PerpScope feature you'll spec later. This manual's job is to make the **data** exist, correct and automated. Phase 6 just makes it fetchable.

Phase 1 — Google Cloud & BigQuery Setup

Goal: an empty but ready warehouse + credentials the pipeline can use.

Steps (you do these once in the browser + gcloud)

1. Create a Google Cloud project, e.g. `perpscope-analytics`.
2. Enable the **BigQuery API**.
3. Create three datasets (region **US** to stay in free tier): `perpscope_raw`, `perpscope_staging`, `perpscope_marts`.
4. Create a **service account** `etl-runner` with roles **BigQuery Data Editor** + **BigQuery Job User**. Download its JSON key **once**.
5. Store that key as a **GitHub Secret** named `GCP_SA_KEY`. For local dev, save it outside the repo and point `GOOGLE_APPLICATION_CREDENTIALS` at it.

ETL/extract/setup_datasets.py — generated in Phase 1

```
from google.cloud import bigquery

client = bigquery.Client() # uses GOOGLE_APPLICATION_CREDENTIALS
for name in ["perpscope_raw", "perpscope_staging", "perpscope_marts"]:
    ds = bigquery.Dataset(f"{client.project}.{name}")
    ds.location = "US"
    client.create_dataset(ds, exists_ok=True)
    print("ready:", name)
```

What this does Connects to BigQuery using your credentials, then loops over the three dataset names and creates each one in the US region. `exists_ok=True` means re-running it is safe — it won't error if a dataset already exists. This is how you spin up the empty warehouse in one command.

Definition of Done: `bq ls` shows the three datasets; `python setup_datasets.py` runs clean; the service-account key is in GitHub Secrets and gitignored locally.

⚠ **Security:** the key file must NEVER be committed. Add `*.json` key patterns to `.gitignore` and only ever pass the key via env var / GitHub Secret.

Phase 2 — Extract (Python)

Goal: for each source, a function that returns a **list of flat dict records** for today, already tagged with `snapshot_date`. No BigQuery yet — extractors just return data (easy to unit-test / print).

2.1 The sources & endpoints

Source	Gives us	Feeds
Hyperliquid — metaAndAssetCtxs	per-asset markPx, oraclePx, openInterest, dayNtIVIm, funding, premium	hottest coins, OI dominance, OI/mcap, OI movers
Hyperliquid — candleSnapshot (1d)	daily OHLCV per asset	strongest coins, 7d price change, regime
Hyperliquid — leaderboard	address + windowed PnL/volume	top-100 traders
Hyperliquid — clearinghouseState	one trader's open positions	positions, net positioning, inflow/outflow
CoinGecko — coins/markets	market cap, symbol, rank	OI/mcap ratio, top-10-by-mcap, symbol mapping
Alternative.me — fng	Fear & Greed index history	F&G chart

Verify-before-trust: endpoint shapes drift. In Phase 2 the very first thing each extractor does is a live call whose JSON we print and eyeball before wiring it in. The Hyperliquid leaderboard host in particular has changed before — we confirm it live.

2.2 Extractor contract (every extractor follows this)

```
# extract/extractors/hyperliquid.py (representative shape)
import requests, datetime as dt

INFO_URL = "https://api.hyperliquid.xyz/info"

def _today() -> str:
    return dt.datetime.now(dt.timezone.utc).date().isoformat()

def fetch_asset_ctxs() -> list[dict]:
    """One record per perp asset: OI, 24h volume, funding, mark/oracle price."""
    r = requests.post(INFO_URL, json={"type": "metaAndAssetCtxs"}, timeout=30)
    r.raise_for_status()
    meta, ctxs = r.json() # [ {universe:[...]}, [ctx, ...] ]
    universe = meta["universe"]
    snap = _today()
    records = []
    for asset, ctx in zip(universe, ctxs):
        records.append({
            "snapshot_date": snap,
            "coin": asset["name"],
            "mark_px": float(ctx["markPx"]),
            "oracle_px": float(ctx["oraclePx"]),
            "open_interest": float(ctx["openInterest"]), # coin units
        })
```

```

        "day_nt1_vlm": float(ctx["dayNt1Vlm"]),          # 24h USD vol
        "funding": float(ctx["funding"]),
        "premium": float(ctx.get("premium") or 0),
    })
    return records

```

What this does

- `fetch_asset_ctxs()` makes one POST call to Hyperliquid asking for every perp asset's live context (price, open interest, volume, funding).
- The response comes back as two parallel lists — asset names and their metrics — so `zip()` pairs them up row by row.
- Each row is turned into a flat dictionary stamped with today's `snapshot_date`, and the numbers are cast to `float` so BigQuery gets clean types. It returns a plain list of dicts — no database writes yet, which makes it trivial to test by printing.

Each extractor exposes small pure functions returning `list[dict]`. `main.py` calls them.

Definition of Done: `python -m extract.main --dry-run` prints record counts and a sample row per source (e.g. "asset_ctxs: 210 rows", "leaderboard: 100 rows", "cg_markets: 250 rows", "feargreed: 1 row"). No BigQuery writes yet.

Phase 3 — Load into BigQuery (raw)

Goal: push each extractor's records into its `perpscope_raw.raw_*` table as a **batch load job**, partitioned by `snapshot_date`, **idempotent** (re-running the same day replaces that day, never duplicates).

3.1 Idempotency strategy: `WRITE_TRUNCATE` per partition

- **Partition decorator load** — load into `raw_table$YYYYMMDD` with `WRITE_TRUNCATE`, which atomically replaces just today's partition. **This is what we use.**
- Alternative: `MERGE` / delete-then-insert — more code, unnecessary here.

3.2 The generic loader

```
# extract/load_bigquery.py (representative)
from google.cloud import bigquery

def load_records(client, dataset, table, records, snapshot_date, schema):
    """Batch-load today's records, replacing only today's partition."""
    if not records:
        print(f"skip {table}: no records"); return
    part = snapshot_date.replace("-", "") # 'YYYYMMDD'
    table_id = f"{client.project}.{dataset}.{table}${part}"
    job_config = bigquery.LoadJobConfig(
        schema=schema,
        write_disposition="WRITE_TRUNCATE", # replace this partition
        time_partitioning=bigquery.TimePartitioning(field="snapshot_date"),
        source_format=bigquery.SourceFormat.NEWLINE_DELIMITED_JSON,
    )
    job = client.load_table_from_json(records, table_id, job_config=job_config)
    job.result()
    print(f"loaded {len(records)} -> {dataset}.{table} [{part}]")
```

What this does

- Takes the list of records an extractor produced and batch-loads them into the matching raw table. If there's nothing to load, it bails early.
- It targets a single day's partition using the `table$YYYYMMDD` suffix, and `WRITE_TRUNCATE` replaces only that partition — so re-running today's job overwrites today's data instead of appending duplicates. That's what makes the pipeline idempotent.
- `time_partitioning` on `snapshot_date` keeps queries cheap (they scan only the days they need), and the newline-delimited-JSON format matches what the extractors emit.

`main.py` wires `extract` → `load` for every source in one run, adding `_loaded_at` and stamping `snapshot_date`.

Definition of Done: after one run, `SELECT snapshot_date, COUNT(*) FROM perpscope_raw.raw_h1_asset_ctxs GROUP BY 1` shows today's row count; re-running the job keeps the count identical (idempotent).

Phase 4 — Transform with dbt (staging → marts)

Goal: the 6 fact tables (feeding all 10 charts), built by dbt, tested, documented. This is the biggest phase; we'll do it in sub-steps: (a) project init + sources, (b) staging, (c) `int_asset_daily` + `dim_asset`, (d) `fct_asset_metrics_daily` + `fct_fear_greed` + `fct_btc_regime`, (e) the Dashboard-3 facts.

4.1 Project config

```
# ETL/dbt/dbt_project.yml
name: perpscope
version: "1.0.0"
profile: perpscope
model-paths: ["models"]
models:
  perpscope:
    staging:      { +materialized: view, +schema: staging }
    intermediate: { +materialized: view, +schema: staging }
    marts:        { +materialized: table, +schema: marts }
```

What this does The main dbt config. The key part is the `models` block: it tells dbt to build the staging and intermediate layers as **views** (cheap, always reflect the latest data) and the marts layer as physical **tables** (fast for Power BI and the JSON export). The `+schema` values route each layer into its own BigQuery dataset.

```
# ETL/dbt/profiles.yml - credentials come from env vars (real key only in CI)
perpscope:
  target: prod
  outputs:
    prod:
      type: bigquery
      method: service-account
      keyfile: "{{ env_var('GOOGLE_APPLICATION_CREDENTIALS') }}"
      project: "{{ env_var('GCP_PROJECT') }}"
      dataset: perpscope_marts          # default; +schema overrides per layer
      location: US
      threads: 4
```

What this does Tells dbt how to connect to BigQuery. Instead of hard-coding secrets, it reads the key file path and project ID from environment variables (`env_var(...)`), so the real credentials only ever live in GitHub Secrets — never in the repo. `threads: 4` lets dbt build independent models in parallel.

4.2 Declare sources (with freshness checks)

```
# models/staging/_sources.yml
version: 2
sources:
  - name: raw
    schema: perpscope_raw
    tables:
      - name: raw_hl_asset_ctxs
        loaded_at_field: _loaded_at
        freshness:
          warn_after: { count: 26, period: hour }
          error_after: { count: 48, period: hour }
          # alert if today's load didn't happen
      - name: raw_hl_candles
      - name: raw_hl_leaderboard
      - name: raw_hl_positions
```

```
- name: raw_cg_markets
- name: raw_feargreed
```

What this does Registers the six raw tables as dbt "sources" so models can reference them and dbt can draw the lineage graph. The **freshness** block on the first table adds a safety net: if the newest row is more than 26 hours old dbt warns, and past 48 hours it errors — a loud signal that a daily extract silently failed to land.

4.3 Staging example (dedupe to latest load per day, cast, rename)

```
-- models/staging/stg_hl_asset_ctxs.sql
with src as (
  select *,
    row_number() over (
      partition by snapshot_date, coin order by _loaded_at desc
    ) as rn
  from {{ source('raw', 'raw_hl_asset_ctxs') }}
)
select
  snapshot_date,
  upper(coin) as coin,
  cast(mark_px as float64) as mark_px,
  cast(open_interest as float64) as open_interest_coins,
  mark_px * open_interest as open_interest_usd, -- notional OI
  cast(day_ntl_vlm as float64) as volume_24h_usd,
  cast(funding as float64) as funding_1h
from src
where rn = 1 -- keep only the freshest load for the day
```

What this does

- A staging model that cleans one raw table without adding business logic. **row_number()** numbers each day's loads per coin from newest to oldest, and **where rn = 1** keeps only the freshest load — so if a day was loaded twice, duplicates disappear.
- The **select** then casts strings to real numeric types, renames columns to clear snake_case, and derives notional open interest (**mark_px * open_interest**). The output is a tidy, typed view that everything downstream can trust.

4.4 The spine: int_asset_daily

One row per (**coin, snapshot_date**) combining ctxs + candle close + CoinGecko mcap. Feeds the wide fact below. (Full SQL delivered in Phase 4c.)

4.5 Facts — clean, reusable, all-assets/all-days (no top-N)

Six facts feed all ten charts; each chart is a **client-side sort** of a fact. The Dashboard-1/2 per-asset metrics are all columns of **one** wide fact — no per-chart tables, no **rank**, no **LIMIT**:

```
-- models/marts/fct_asset_metrics_daily.sql (one wide fact -> 5 charts)
with d as (select * from {{ ref('int_asset_daily') }}),
btc as (select snapshot_date, close_px as btc_close_px from d where coin='BTC'),
w as (
  select
    d.coin, d.snapshot_date, d.mark_px, d.close_px, d.volume_24h_usd,
    lag(d.volume_24h_usd,1) over (partition by d.coin order by d.snapshot_date) as
    volume_24h_usd_prev,
```

```

        d.open_interest_usd,
        lag(d.open_interest_usd,1) over (partition by d.coin order by d.snapshot_date) as
oi_usd_prev,
        lag(d.open_interest_usd,7) over (partition by d.coin order by d.snapshot_date) as
open_interest_usd_7d_ago,
        lag(d.close_px,7) over (partition by d.coin order by d.snapshot_date) as
close_px_7d_ago,
        d.market_cap_usd, d.funding_1h,
        d.close_px / b.btc_close_px as coin_btc_ratio,
        lag(d.close_px / b.btc_close_px,30) over (partition by d.coin order by d.snapshot_date) as
coin_btc_ratio_30d_ago
    from d join btc b using (snapshot_date)
)
select
    w.*,
    safe_divide(volume_24h_usd - volume_24h_usd_prev, volume_24h_usd_prev) as
vol_change_24h_pct,
    open_interest_usd / sum(open_interest_usd) over (partition by snapshot_date) as
oi_dominance,
    safe_divide(open_interest_usd - oi_usd_prev, oi_usd_prev) as
oi_change_24h_pct,
    open_interest_usd - open_interest_usd_7d_ago as
oi_change_7d_usd,
    safe_divide(close_px, close_px_7d_ago) - 1 as
price_change_7d_pct,
    safe_divide(open_interest_usd, market_cap_usd) as
oi_mcap_ratio,
    safe_divide(coin_btc_ratio, coin_btc_ratio_30d_ago) - 1 as rs_30d,
    a.market_cap_rank, a.is_stablecoin, a.is_wrapped
from w left join {{ ref('dim_asset') }} a using (coin)
-- No WHERE latest, no ORDER BY, no LIMIT: the app/Power BI slice this.

```

What this does

- The one wide fact that powers five different charts. The **btc** CTE pulls BTC's daily close so every coin can be measured relative to Bitcoin.
- The **w** CTE uses **lag(...)** window functions to look back 1, 7, and 30 days per coin — that's how it gets "yesterday's" and "a week ago's" values needed for change calculations, all in one pass.
- The final **select** turns those into the metrics the dashboards read: 24h volume change, OI dominance (a coin's share of total open interest that day), 7-day OI and price change, the OI-to-market-cap ratio, and 30-day relative strength. **safe_divide** returns null instead of crashing on divide-by-zero.
- Crucially it has **no `ORDER BY`, `LIMIT`, or top-N filter** — it holds every coin on every day, and the app or Power BI does the sorting and "top 5" slicing. One fact, many charts, zero schema changes when a chart's needs change.

fct_asset_positioning_daily follows the same shape for the smart-money cohort (per-coin long/short/flat counts + signed net-notional day-over-day delta), and **fct_fear_greed** / **fct_btc_regime** / **fct_trader_daily** / **fct_trader_positions** are the four thin facts.

4.6 Tests

In **_marts.yml** / **_staging.yml** add dbt tests: **not_null** + **unique** on grain keys (**coin** + **snapshot_date**), **accepted_range** (e.g. **oi_dominance** 0–1; **pct_long+pct_short+pct_flat** = 1), **relationships** from positions → **dim_asset**, and a reconciliation test that **sum(oi_dominance)** ≈ 1 per day. **dbt build** runs models **and** tests.

Definition of Done: `dbt build` succeeds; `dbt docs generate` produces the lineage graph; all 6 fact tables (+ `dim_asset`) exist in `perpscope_marts` with sane numbers spot-checked against the live app.

Data Dictionary — What the Fact Tables Look Like

Phase 4 shows the full SQL for only `fct_asset_metrics_daily`. This section fills in the rest: the **column-level shape** of all six facts (plus `dim_asset`), so you can see exactly what each table stores and which chart each column feeds. This is the single source of truth — a mart's SQL should always match its entry here.

Design principle — clean reusable tables; the app/BI sorts and slices. Each fact holds *every* row at its grain (all assets, all days) plus every column a component might sort or filter on. **No `rank`, no `LIMIT`, no `is\_latest`, no top-N in the warehouse** — ordering and slicing are presentation concerns. That's why the per-asset metrics for Dashboards 1 & 2 all live as columns of **one** wide fact, and the ten charts need just **six facts + `dim\_asset`**.

dim_asset — the asset dimension

dim_asset

Feeds: every per-asset fact joins to it

Grain: one row per asset (`coin`)

Column	Meaning
coin	Uppercased Hyperliquid perp symbol (e.g. BTC). Primary key.
cg_id	CoinGecko id mapped from the symbol (for market-cap joins).
name	Human-readable name from CoinGecko.
market_cap_rank	Latest CoinGecko rank; denormalized into facts so "top-10 by mcap" needs no join.
is_stablecoin	TRUE if in the stablecoin exclude-list (USDT/USDC/DAI...).
is_wrapped	TRUE if a wrapped/staked derivative (WBTC/WETH/stETH...).

Why it matters: the symbol → CoinGecko mapping is the one fuzzy join in the project (Hyperliquid uses `BTC`, CoinGecko uses `bitcoin`), so `dim_asset` holds an override map. The stablecoin/wrapped flags let every fact filter those out consistently.

fct_asset_metrics_daily — the wide per-asset fact

fct_asset_metrics_daily

Feeds: 5 charts across Dashboards 1 & 2 (hottest, strongest, OI dominance, OI 7d movers, OI/mcap)

Grain: one row per (coin, snapshot_date) — all assets, all days

Column	Meaning	Used by
coin, snapshot_date	keys	all
mark_px, close_px	price at snapshot / daily close	—
volume_24h_usd	trailing-24h notional volume	hottest
volume_24h_usd_prev	prior day's volume (lag)	hottest (raw input)
vol_change_24h_pct	$(\text{volume} - \text{volume_prev}) / \text{volume_prev}$	hottest coins (sort desc)
open_interest_usd	$\text{OI\$} = \text{open_interest_coins} \times \text{mark_px}$	dominance, movers, oi/mcap
oi_dominance	$\text{OI\$} / \text{sum}(\text{OI\$})$ that day (0–1, sums to 1)	OI dominance treemap (area)
oi_change_24h_pct	$(\text{OI\$_today} - \text{OI\$_yesterday}) / \text{OI\$_yesterday}$	OI treemap (color)
open_interest_usd_7d_ago	OI\$ 7 days ago (lag 7)	movers (raw input)
oi_change_7d_usd	$\text{open_interest_usd} - \text{open_interest_usd_7d_ago}$	OI 7d movers (sort desc)
oi_change_7d_pct	same, as % of 7d-ago	OI 7d movers
price_change_7d_pct	$(\text{close_px} / \text{close_px_7d_ago}) - 1$	OI 7d movers
market_cap_usd	spot market cap (CoinGecko)	oi/mcap
oi_mcap_ratio	$\text{open_interest_usd} / \text{market_cap_usd}$ — perp leverage vs spot	OI/mcap bar (sort desc)
coin_btc_ratio	$\text{close_px} / \text{btc_close_px}$	strongest (raw input)
coin_btc_ratio_30d_ago	same ratio 30 days ago	strongest (raw input)
rs_30d	$(\text{coin_btc_ratio} / \text{ratio_30d_ago}) - 1$ — 30d outperformance vs BTC	strongest coins (sort desc)
funding_1h	current 1h funding rate	bonus / tooltips
market_cap_rank	CoinGecko rank (from dim_asset)	any top-N by mcap
is_stablecoin, is_wrapped	exclusion flags (from dim_asset)	client filters out

⚠ Metric correction — `rs\_30d` (worth telling in an interview). The spec said "highest avg coin/btc ratio over 30d." Taken literally that ranks by the *price level* of coinPx/btcPx, which is scale-dependent

and meaningless. "Strength" must measure **outperformance vs BTC**, so `rs_30d` stores the 30-day *change* in the coin/BTC ratio: +0.25 = beat BTC by 25%.

fct_fear_greed — the two thin Dashboard-1 facts

fct_fear_greed

Feeds: Fear & Greed gauge + line chart (Dashboard 1)

Grain: one row per `date`

Column	Meaning
date	Index date (Alternative.me).
fng_value	0–100 Fear & Greed index.
fng_classification	Extreme Fear ... Extreme Greed.

Pass-through of the full Fear & Greed history in one call, deduped by date. The "current" gauge is just the client reading the max `date` — no `is_latest` flag needed.

fct_btc_regime

fct_btc_regime

Feeds: BTC bull/bear regime badge + history (Dashboard 1)

Grain: one row per `snapshot_date` (BTC only)

Column	Meaning
snapshot_date	Day.
regime	Bull / Bear / Range / Accumulation.
regime_confidence	Posterior probability of the assigned state (0–1).
model_version	Version tag of the HMM params used (auditability across retrains).

Reuses the existing HMM regime model: a Python step computes the regime features from BTC daily OHLCV + funding, runs the trained HMM, and lands (`regime`, `confidence`) per day. Warm-up: `regime` = `'Unknown'` until enough history exists.

fct_asset_positioning_daily — smart-money positioning

fct_asset_positioning_daily

Feeds: net positioning stacked bar + daily inflow/outflow (Dashboard 3)

Grain: one row per (coin, snapshot_date) — every asset the top-100 cohort holds

Column	Meaning	Used by
coin, snapshot_date	keys	both
cohort_size	# traders in the cohort that day (100; stored so pct_* is auditable)	both
n_long	# cohort traders net-long this coin	net positioning
n_short	# cohort traders net-short	net positioning
n_flat	cohort_size – n_long – n_short (not involved)	net positioning
pct_long	n_long / cohort_size	net positioning (sort desc)
pct_short	n_short / cohort_size	net positioning
pct_flat	n_flat / cohort_size	net positioning
cohort_net_notional_usd	sum(signed_notional_usd) across cohort (+ = net-long)	inflow
cohort_net_notional_prev	same, yesterday (lag)	inflow (raw input)
inflow_usd	net_notional – net_notional_prev (+ = added long / inflow)	inflow/outflow
inflow_pct	inflow_usd / abs(cohort_net_notional_prev)	inflow/outflow
market_cap_rank	from dim_asset — lets client show top-10 by mcap	inflow/outflow filter

The denominator is the **full cohort (100)**, so $pct_long + pct_short + pct_flat = 1$ and "not involved" is meaningful — that's the whole point of the stacked bar. **Cohort-churn gotcha:** membership changes daily, so prefer a fixed-cohort diff for **inflow_usd** (only addresses in *both* days' cohorts) and expose churn separately.

fct_trader_daily — the two trader-level facts

fct_trader_daily

Feeds: top-100 trader stats table (Dashboard 3)

Grain: one row per (trader_address, snapshot_date)

Column	Meaning
trader_address	Wallet.
snapshot_date	Day.
pnl_30d_usd	30-day PnL (leaderboard month window) — client sorts by this.
volume_30d_usd	30-day traded volume.
account_value_usd	Current account value.
n_open_positions	Count of open positions that day (0 for all-cash traders).
gross_exposure_usd	sum(position_value_usd) across positions.
net_exposure_usd	sum(signed_notional_usd) (long – short).
in_cohort	TRUE if in the top-100-by-30d-PnL set that day (definitional filter).

The client filters `in_cohort` and sorts by `pnl_30d_usd` — no stored `rank` needed. Some top traders hold zero positions (all-cash); that's expected.

fct_trader_positions

fct_trader_positions

Feeds: expandable positions detail per trader; source for fct_asset_positioning_daily

Grain: one row per (trader_address, coin, snapshot_date)

Column	Meaning
trader_address, coin, snapshot_date	keys
size_coins	Signed position size (szi): + = long, – = short.
position_value_usd	abs(size_coins) × mark_px (notional).
signed_notional_usd	size_coins × mark_px (signed: + long / – short).
entry_px	Average entry price.
unrealized_pnl	Unrealized PnL reported by Hyperliquid.
leverage	Position leverage.
direction	'long' if szi > 0, 'short' if szi < 0.

Chart → table map

Every chart is a **sort/filter of one clean table** — no chart-specific tables exist. This is the payoff of the wide-fact design:

Dashboard	Chart	Table	Client does
1	BTC regime badge	fct_btc_regime	latest row
1	Fear & Greed gauge + line	fct_fear_greed	latest + time series
1	Hottest 5 coins	fct_asset_metrics_daily	sort vol_change_24h_pct desc, take 5
1	Strongest 5 coins	fct_asset_metrics_daily	sort rs_30d desc, take 5
2	OI dominance treemap	fct_asset_metrics_daily	oi_dominance (area) + oi_change_24h_pct (color)
2	Top OI movers (7d)	fct_asset_metrics_daily	sort oi_change_7d_usd desc, take 5
2	OI/mcap leverage bar	fct_asset_metrics_daily	sort oi_mcap_ratio desc, take 5
3	Top-100 trader stats	fct_trader_daily (+positions)	filter in_cohort, sort pnl_30d_usd desc
3	Net positioning bar	fct_asset_positioning_daily	sort pct_long desc, take 10
3	Daily inflow/outflow	fct_asset_positioning_daily	filter market_cap_rank ≤ 10, show inflow_usd

Cross-cutting gotchas

- **Units:** Hyperliquid `openInterest` is in **coin units** — always multiply by `mark_px` for USD. `dayNt1V1m` is already USD notional; `sz_i` is signed size in coin units.
- **OI history is not backfillable** (HL exposes only current OI) — `oi_change_24h_pct` and `oi_change_7d_usd` are NULL until the pipeline has accumulated 2 / 8 daily snapshots. Start the daily job early.
- **Timezone:** everything keys on UTC `snapshot_date`; keep the cron hour fixed so `lag`-based math stays consistent.
- **History depth:** BigQuery sandbox expires partitions at 60 days. 30d-lookback metrics work fine; for >60d history attach billing or export raw to GCS as Parquet.

Phase 5 — Orchestrate Daily with GitHub Actions

Goal: one workflow that, every day, runs extract+load, then **dbt build**, then the JSON export — with the service-account key injected from Secrets.

```
# .github/workflows/market_analytics_etl.yml
name: Market Analytics ETL
on:
  schedule:
    - cron: "0 6 * * *"      # 06:00 UTC daily
  workflow_dispatch: {}      # manual "Run" button for testing

jobs:
  etl:
    runs-on: ubuntu-latest
    env:
      GCP_PROJECT: ${ secrets.GCP_PROJECT }
      GOOGLE_APPLICATION_CREDENTIALS: ${ github.workspace }/sa_key.json
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }

      - name: Write service-account key from secret
        run: echo '${ secrets.GCP_SA_KEY }' > "$GOOGLE_APPLICATION_CREDENTIALS"

      - name: Install deps
        run: |
          pip install -r ETL/extract/requirements.txt
          pip install dbt-bigquery

      - name: Extract + Load
        run: python -m extract.main
        working-directory: ETL

      - name: Transform (dbt)
        run: dbt build --project-dir ETL/dbt --profiles-dir ETL/dbt

      - name: Export marts -> JSON
        run: python -m export.export_marts
        working-directory: ETL

      - name: Publish JSON to GCS
        run: gsutil -m cp -r ETL/export/out/*.json gs://perpscope-public/market/
```

What this does

- The daily automation. **on.schedule.cron** fires it at 06:00 UTC every day, and **workflow_dispatch** adds a manual "Run" button for testing.
- It writes the service-account key from a GitHub Secret into a temp file, installs dependencies, then runs the four stages in order: extract + load → **dbt build** (which runs models **and** tests) → export marts to JSON → publish the JSON to the public bucket.
- Because the key comes from Secrets and the whole thing lives in the repo, this is your CI/CD + scheduling story in one file — no external orchestrator needed.

Notes

- The cron is UTC. Pick a time after your sources have finalized the prior UTC day.
- **workflow_dispatch** lets you click **Run workflow** to test without waiting for the cron.

- Free minutes: this job runs ~2–3 min/day → trivial against the 2000 min/month private limit (unlimited if the repo is public).

Definition of Done: a manual `workflow_dispatch` run goes green end-to-end; the next morning it fires on its own and marts show the new `snapshot_date`.

Phase 6 — Serve to the React App (static JSON export)

Goal: turn the finished marts into small JSON files the browser can `fetch()` — keyless, free, cached, exactly like PerpScope's other features. **This is where "SQL for the app" lives:** the export runs `SELECT * FROM <mart>` and writes JSON.

```
# export/export_marts.py (representative)
import json, pathlib
from google.cloud import bigquery

OUT = pathlib.Path("export/out"); OUT.mkdir(parents=True, exist_ok=True)
client = bigquery.Client()

FACTS = [
    "fct_asset_metrics_daily", "fct_fear_greed", "fct_btc_regime",
    "fct_asset_positioning_daily", "fct_trader_daily", "fct_trader_positions",
    "dim_asset",
]
for m in FACTS:
    rows = [dict(r) for r in client.query(
        f"SELECT * FROM `{client.project}.perpscope_marts.{m}`").result()]
    (OUT / f"{m}.json").write_text(json.dumps(rows, default=str))
    print(f"exported {len(rows)} rows -> {m}.json")
```

What this does Turns the finished marts into static files the React app can fetch. It loops over the six facts plus `dim_asset`, runs a plain `SELECT *` on each, and writes the rows to a `.json` file (`default=str` safely serializes dates and other non-JSON types). This is the whole "SQL for the app" layer — no API server, just cached JSON the browser reads directly.

Two ways to publish (pick one; recommended = GCS bucket)

- **Public GCS bucket** `gs://perpscope-public/market/*.json` → React fetches the file URL. Clean, no repo noise, CDN-cached.
- **Commit into the repo** under `src/features/market/data/` → simplest, but adds a daily auto-commit. Fine for a résumé project if you'd rather avoid a bucket.

The React side later just does `fetch(url)` and renders with Recharts (treemap, stacked bar, etc.) — same pattern as your existing features.

Definition of Done: the fact JSON files are reachable at a stable public URL and contain the full history (the app slices to the latest day itself).

Phase 7 — Power BI Dashboard

Goal: a Power BI report reading the **same marts** live from BigQuery.

Steps

6. Power BI Desktop → **Get Data** → **Google BigQuery** → sign in.
7. Select `perpscope_marts` → load the mart tables (or write a custom SQL query).
8. Build the three report pages mirroring the three dashboards (treemap for OI dominance, stacked bar for net positioning, cards for F&G, etc.).
9. Publish to Power BI Service → configure **scheduled refresh** (daily, after the ETL cron).

Modeling tip: relate every fact table to `dim_asset` on `coin` in the Power BI model view, so slicers (by asset) work across visuals. Because marts are pre-aggregated to chart grain, DAX stays minimal — the heavy lifting was done in dbt (that's the point of the marts layer).

Definition of Done: three-page report refreshes on schedule from BigQuery; numbers match the React app and your spot-checks.

8. Data Quality, Testing & Monitoring

- **dbt tests** (Phase 4.6): schema tests on every grain key; range tests on ratios/percents.
- **Source freshness** (`dbt source freshness`): fails loudly if a day's extract didn't land.
- **CI on pull requests**: a second lightweight GitHub Actions workflow runs `dbt build --select state:modified+` against a scratch dataset so model changes are tested before merge.
- **Idempotency**: partition-truncate loads mean re-runs never duplicate.
- **Backfill**: because raw is retained and partitioned, you can re-run dbt for any historical range without re-hitting APIs.
- **Alerting (optional)**: pipe a failure notification from the workflow to email/Discord.

9. Cost & Free-Tier Limits

Resource	Free limit	Our usage	Verdict
BigQuery storage	10 GB	~KBs/day (few MB/year)	nowhere close
BigQuery query	1 TB scanned/month	partitioned; MBs/run	nowhere close
BigQuery sandbox	60-day partition expiry, no streaming	we batch-load, set expirations	fine
GitHub Actions	2000 min/mo (private), ∞ (public)	~3 min/day ≈ 90 min/mo	fine
GCS bucket	5 GB free tier	KBs of JSON	fine
CoinGecko / Alt.me / Hyperliquid	public, rate-limited	≤ a few hundred calls/day	within limits

Bottom line: the whole pipeline runs at **\$0** at this data volume. The only paid path is if you attach a BigQuery billing account for >60-day history — and even then you'd stay inside the perpetual free tier.

10. Appendix: the Metric / Data Dictionary

The exact math for every chart — sources, grain, formula, and edge cases — lives in [ETL/docs/DATA_DICTIONARY.md](#). **Read that before building any mart**; it's where the analytical decisions (e.g. the "strongest coins" redefinition) are justified. A mart's SQL should always match its dictionary entry.

How to use this manual from here

Prompt me phase by phase, e.g. "**Do Phase 1**", then "**Build the extractors**", etc. For each phase I'll deliver the real files, we'll run them, and we'll confirm the **Definition of Done** before advancing. Start whenever you're ready.